

Cascading Style Sheet (CSS)

Lesson 00: (version 3.0)

IGATE is now a part of Capgemini

People matter, results count.

Capgemini Public



Copyright © 2011 IGATE Corporation (a part of Capgemini Group). All rights reserved.

No part of this publication shall be reproduced in any way, including but not limited to photocopy, photographic, magnetic, or other record, without the prior written permission of IGATE Corporation (a part of Capgemini Group).

IGATE Corporation (a part of Capegemini Group) considers information included in this document to be confidential and proprietary.

Document History

Add instructor notes
here.

Date	Course Version No.	Software Version No.	Developer / SME	Change Record Remarks
10-Dec-2012	1.0	3.0	Mohan Chinnaiyah	
31-Mar-2015	2.0	3.0	Rathnajothi Perumalsamy	Changes made for aligning to the upgraded ELTP course structure.
May - 2016	2.1	3.0	Anjulata	Changes made for aligning to the upgraded ELTP course structure.
July – 2020	2.2	3.0	Neelima	Revamp as per 2020 ToC

Course Goals and Non Goals

➤ Course Goals

- Understand Cascading Style Sheet 3.0
- Understand new features of CSS 3.0
- Understanding RGBA and HSLA color schemes



➤ Course Non Goals

- Animation
- Be able to use Custom Fonts
- Exploring Layouts supported by CSS 3.0

Pre-requisites

- HTML 5
- Understanding of Different Browsers like IE , FireFox, Opera and Chrome

Intended Audience

➤ Web page designers



Day Wise Schedule

➤ Day 1

- Lesson 1: Introduction to CSS 3
- Lesson 2: Working with Text and Fonts
- Lesson 3: CSS Selectors

Table of Contents

➤ Lesson 1: Introduction to CSS 3

- 1.1. Introduction
- 1.2. CSS Syntax
- 1.3. Types of CSS

➤ Lesson 2: Working with Text and Fonts

- 2.1. Text Formatting
- 2.2. Text Effects
- 2.3. Fonts

➤ Lesson 3: CSS Selectors

- 3.1 Introduction to Selectors
- 3.2. Universal Selector
- 3.3 Type Selector
- 3.4 Class Selector
- 3.5 ID Selector
- 3.6 Attribute Selector
- 3.7 PseudoClasses

References

- W3 Schools
- Sitepoint



Software required

- Editor like notepad, notepad++
- Browsers (IE, Google Chrome, FireFox and Opera)

Add instructor notes
here.

Cascading Style Sheet 3.0

Lesson 01: Introduction to CSS 3.0

Lesson Objectives

- In this lesson, we will learn:

- Introduction to CSS
 - What is CSS
 - CSS History
 - CSS 3.0 features
 - What CSS can do
- CSS Syntax
- Types of CSS



Explain the lesson coverage

What is CSS ?

- Cascading Style Sheets (CSS) is a style sheet language used to describe the presentation (that is, the look and formatting) of a document written in a markup language.
- CSS was created by Hakon Wium Lie and Bert Bos and was adopted as a W3C Recommendation in late 1996

Cascading Style Sheet:

- CSS allows complete and total control over the style of a hypertext document
- A standards-based method for controlling the look and feel of HTML content.
- Comprised of Rules to control elements in the document.
- Designed to separate formatting from the content while being flexible and scalable

What is a Style Sheet?

- Style sheets define how to display HTML elements.
- Style sheets (SS) provide a means for web authors to separate the appearance of web pages from the content.
- Style sheets are an accepted standard on the W3C. The standards are referred to as Cascading Style Sheets 1 (CSS1) and Cascading Style Sheets 2 (CSS2).

CSS History

Version	Description	Features
CSS 1	The first CSS specification, an official W3C Recommendation, published in December 1996	typeface, emphasis, backgrounds, spacing between words, letters, and lines of text. Alignment of text, images, tables and other elements Margin, border, padding etc
CSS 2	CSS level 2 specification was developed by the W3C and published as a recommendation in May 1998.	includes a number of new capabilities like absolute, relative, and fixed positioning of elements and z-index, the concept of media types, support for aural style sheets and bidirectional text, and new font properties such as shadows
CSS2.1	CSS 2.1 was published as a W3C Recommendation on 7 June 2011	CSS level 2 revision 1, often referred to as "CSS 2.1", fixes errors in CSS 2, removes poorly supported or not fully interoperable features and adds already-implemented browser extensions to the specification
CSS 3	Current version	CSS 3 is divided into several separate documents called "modules". Each module adds new capabilities or extends features defined in CSS 2. As of June 2012, there are over fifty CSS modules published from the CSS Working Group

Capgemini Public

CSS 1 Features:

- [Font](#) :properties such as typeface and emphasis
- Color of text, backgrounds, and other elements
- Text attributes such as spacing between words, letters, and lines of text
- [Alignment](#) of text, images, [tables](#) and other elements
- Margin, border, padding, and positioning for most elements
- Unique identification and generic classification of groups of attribute

CSS 2 Features:

CSS level 2 specification was developed by the W3C and published as a recommendation in May 1998. A superset of CSS 1, CSS 2 includes a number of new capabilities like absolute, relative, and fixed positioning of elements and z-index, the concept of media types, support for aural style sheets and bidirectional text, and new font properties such as shadows.

CSS 2.1 Features:

CSS level 2 revision 1, often referred to as "CSS 2.1", fixes errors in CSS 2, removes poorly supported or not fully interoperable features and adds already-implemented browser extensions to the specification

Why CSS?

➤Solves common problem:

- Separate document presentation from the web page content.

➤Save lots of work:

- Allows developers to control the style and layout of multiple Web pages all at once.

Why use CSS?

Styles solve a common problem : HTML tags were originally designed to define the document content. They were supposed to say "This is a header", "This is a paragraph", "This is a table", by using tags like <h1>, <p>, <table>, and so on. Browser was to take care of the layout of the document without using any formatting tags.

Two major browsers - Netscape and Internet Explorer - continued to add new HTML tags and attributes (like the tag and the color attribute) to the original HTML specification. Subsequently, it became more difficult to create HTML documents with content clearly separate from the presentation layout. To solve this problem, W3C, the non-profit, standard setting consortium responsible for standardizing HTML, created STYLES in addition to HTML.

Style Sheets Save a Lot of Work

Styles in HTML define how HTML elements are displayed, just like the *bold tag*. Styles are saved in files external to your HTML documents. External style sheets allow you to change the appearance and layout of all pages in your website. Simply, edit a single CSS document. If you have ever had to change the font or color of all the headings in all your Web pages, you will understand how CSS can save you a lot of work.

CSS is a breakthrough in Web design because it allows developers to

control the style and layout of multiple Web pages all at once. As a Web developer you can define a style for each HTML element and apply it to as many Web pages as you want. To make a global change, simply change the style, and all elements in the Web are updated automatically.

CSS 3.0 Features

- Many exciting new functions and features have been introduced in CSS3.
- Following table list some of the new features

Property	New Attributes			
Borders	border-color	border-image	border-radius	box-shadow
Backgrounds	background-origin	background-size	multiple-backgrounds	
Color	HSL Colors	HSLA Colors	RGBA Colors	opacity
Text Effects	text-shadow	text-overflow	word-wrap	
Selectors	Attribute-selector	:nth-child()	:nth-of-type()	

- Many more features like...
 - CSS3 Transitions
 - Animations
 - media queries
 - multi-column layout
 - Web fonts

Capgemini Public

CSS 3 has introduced many features using which we can much more powerful and flexible websites. Some of the new features of CSS 3 are as follows:

Border Radius: Creating rounded corners in web design isn't always the easiest of things to accomplish. Thanks to the power of CSS3, it has since become one of the more popular and easier techniques to implement. By taking advantage of the **border-radius** property, you can easily round off those corners in seconds

Box Shadow: Creating box shadows is another pretty cool example for adding some stylish elements to your web designs. The best part being the fact that it is completely executed without the use of images. There are even ways to add multiple box shadows to your rounded corners, allowing for the possibility of creating some very slick designs

Multiple Background Images: Another cool example of CSS3 is the ability to apply multiple backgrounds to a single DIV without having to create extra child DIV's whose only purpose is to support an image

Text Shadow: You know how easy it is to double click a layer in Photoshop and say hey, I want to add a quick drop shadow to that

text? This may be even easier than that. You aren't just restricted to just one shadow either. By combining multiple text shadows of varying colors, the possibilities are endless.

@Font-Face: With this feature we can include custom fonts into our web pages. We can now begin to take advantage of various other fonts, whether installed on the readers computer or not, assuming that they can be pulled via an online directory. Just upload the desired font to your server and pull it via the @font-face feature.

Multi-column layout: W3C offers a new way to arrange text “news-paper wise”, in columns. [Multi-column layout](#) is actually a module on its own. It allows a web developer to let text be fitted into columns

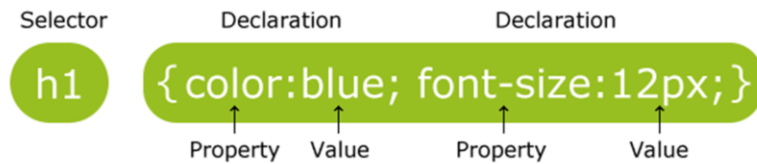
CSS3 Animations: Traditionally, the Web was a very static place. Achieving animations was not really possible unless we use JavaScript, animated GIFs and Flash . But With CSS3, we can create animations, which can replace animated images, Flash animations, and Java Scripts in many web pages.

What Can CSS Do?

- Text formatting
- Element sizing
- Element positioning
- Change link attributes
- Cursor manipulation
- Animation

Many More....

CSS Syntax



- A CSS rule has two main parts:
 - A selector
 - One or more declarations
- The selector is normally the HTML element you want to style.
- Each declaration consists of a property and a value.
- The property is the style attribute you want to change. Each property has a value.

Capgemini Public

A CSS declaration always ends with a semicolon, and declaration groups are surrounded by curly brackets:

```
p {color:red;text-align:center;}
```

To make the CSS more readable, you can put one declaration on each line, like this:

```
p
{
color:red;
text-align:center;
}
```

Types of CSS

- Three CSS implementations
 - Inline
 - Affects only the element applied to
 - Embedded
 - Affects only the elements in a single file
 - External
 - Linked to an unlimited number of files

Types of CSS:

- Inline: Style sheet definition only applies to the tag contents that contain it. It is used to control a single tag element. Each tag does not need to have its style defined as it inherits from its parent.
- Embedded: Embedded style sheets are placed within HTML code of the page they are to be applied to. Style sheet syntax comes between opening and closing <STYLE> tags. These tags are placed either in the <HEAD> section or between the </HEAD> and <BODY> tags.
- Linked: Linked style sheets exist as separate files that are linked to a page with the <LINK> tag. They have the css extension and are referenced with a URL. Inside the css file, style attributes are contained within opening and closing <STYLE> tags. Placing a single <LINK> tag within the <HEAD> tags links the page that needs these styles.

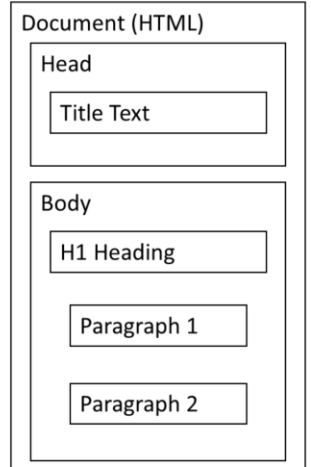
HTML Page Structure

```
<!DOCTYPE HTML>
<HTML>

  <HEAD>
    <TITLE>Title Text</TITLE>
  </HEAD>

  <BODY>
    <H1>H1 Heading</H1>
    <P>Paragraph 1</P>
    <P>Paragraph 2</P>
  </BODY>

</HTML>
```



The above given HTML document content is not formatted using CSS.

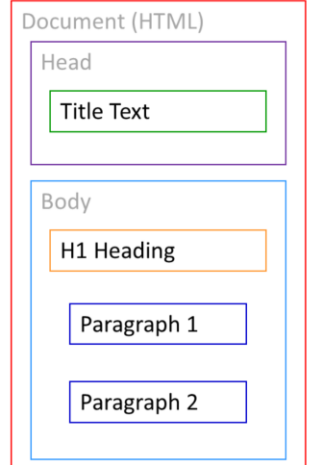
HTML Page Structure with CSS

```
<!DOCTYPE HTML>
<HTML>

  <HEAD>
    <TITLE>Title Text</TITLE>
  </HEAD>

  <BODY>
    <H1>H1 Heading</H1>
    <P>Paragraph 1</P>
    <P>Paragraph 2</P>
  </BODY>

</HTML>
```



The above given HTML document content is formatted using CSS.

Inline CSS

- **Inline Style Sheets:**
 - All style attribute are specified in the tag it self.
 - It gives desired effect on that tag only. It does not affect any other HTML tag.
- **Syntax:**

```
<element style="propertyname : value; propertyname : value">
```

- **An example of STYLE attribute usage:**

```
<p style="font-weight: bold">This is bold text</p>
```

```
<p style="font-weight: bold">This is bold text</p>
```

is equivalent to

Capgemini Public

Inline Style Sheet:

Definitions appear next to other tag attributes. You need to remember to place the style sheet description within quotes, like the following:

```
<!DOCTYPE HTML>
<html>
<head><title>Inline Style Sheet</title></head>
<body style="background: white; color:green">
<h2 style="background: gold; font-family: Arial, Impact, Sans Serif;
color:red">
This is Level 2 Heading, with style</h2>
<h1 style="background: orange; font-family: Arial, Impact, Sans serif;
color: blue;font-size:30pt; text-align: center">
This is Level 1 Heading, with style</h1>
<h3 style="background: gold; font-family: Arial, Impact, Sans Serif;
color:red">
This is Level 3 Heading, with style</h3>
<h4>This is Level 4 Heading, without style</h4>
<h1>This is again Level 1 heading with default styles</h1>
</body>
</html>
```

Embedded CSS

- **Embedded Style Sheet:**
 - Set of style definitions placed within <STYLE> tags.
 - Added to the <HEAD> area of file

- **Syntax:**

```
<HEAD>
    <STYLE TYPE="text/css">...</STYLE>
</HEAD>
```

- **An example of <STYLE> tag usage:**

```
<HEAD>
    <TITLE>New Topic</TITLE>
    <STYLE>P {font-weight : bold}</STYLE>
</HEAD>
```

Capgemini Public

Embedded Style Sheet:

An embedded style sheet is a set of style definitions placed within <STYLE> tags and located in the HEAD section of the HTML document. It sets the style attributes for the entire page where it is located. Following style sheet description applies to the <H1> tag. It sets the font face to be either Arial, Impact, or Sans Serif, depending on which one it finds first on the user's system. Text color is also defined as blue.

H1 {font-family: Arial, Impact, Sans Serif; color: blue}

You can also group tags together by separating them with commas:

```
H1, H2, H3 {font-family: Arial, Impact, Sans Serif; color: blue}<html>
<html>
<head>
<style>  body {background: black; color:green}
h1 {background: orange; font-family: Arial; color:blue}
h2, h3 {background: gold; font-family: Arial, Impact, Sans Serif; color:red}
</style> </head>
<body>
<h2>This is Level 2 Heading, with style</h2>
<h1>This is Level 1 Heading, with style</h1>
<h3>This is Level 3 Heading, with style</h3>
<h4>This is Level 4 Heading, without style</h4>
</body>
</html>
```

External CSS

- The <LINK> element is used to attach an external CSS document to an HTML document
 - All style definition are store in one file (.css file)
 - This file gets called by the HTML file during page loading
 - **Syntax:** <link rel="stylesheet" href="filename.css" type="text/css">

- **Example**

– Content in first.css:

```
{font-weight : bold}

<HEAD>
  <TITLE>Demo CSS</TITLE>
```

– Content in first.html file:

```
<LINK REL="stylesheet" href="first.css" REL="stylesheet" TYPE="text/css">
</HEAD>
```

Capgemini Public

External CSS

External CSS is same as embedded style sheet. The only difference is that the separate css file contains all styles, and gets called by the HTML file.

Example:

```
<!DOCTYPE HTML>
<html>
<head>
<title>Linked Style Sheet</title>
<link rel=stylesheet href="linked_ex2.css" type="text/css">
</head>
<body>
<h2>This is Level 2 Heading, with style</h2>
<h1>This is Level 1 Heading, with style</h1>
<h3>This is Level 3 Heading, with style</h3>
<h4>This is Level 4 Heading, without style</h4>
</body></html>
```

CSS Precedence

- **Browser determines default format.**
- **Order of precedence when three CSS types combine at run time in the HTML page are:**
 - Inline styles
 - Embedded style sheets
 - Linked (external) style sheets

Style Sheet Precedence

There are several rules that apply to the order of precedence of style sheets. All tags have a default format determined by the browser. This is what you see if no style sheet attributes are set. This also represents the lowest priority.

Another level of priority is established by how close the style definition is to the tag. For this order, linked style sheets are lower than embedded style sheets, which are lower than inline style sheets. If you accidentally include the same property in a linked style sheet as in inline style sheet, then the priority goes to the definition closest to the tag, which would be inline style.

Style sheets for more specific tags have priority over general tags. For example, if a Web page marks the <BODY> tag with a certain style sheet definition and an <H3> tag with same property and a different value, then the <H3> tag has the priority, even though it is also part of the body.

Demo : CSS Syntax and CSS Types

- Lesson01

- demo1.html
- Embeddedstylesheet.htm
- Linkedstylesheet.htm
- Inlinestylesheet.htm



Lesson Summary

- In this lesson, you have learnt about:

- What is CSS
- CSS history
- What CSS can do
- CSS Syntax
- Types of CSS



Additional notes for
instructor

Review Questions

Ans-1 : 4
Ans-2 : 1,2

- Question 1: Which of the following are CSS Types.
 - Inline
 - Embedded
 - External
 - All the above
- Question 2: CSS rule has _____ and _____
 - Selector
 - Declaration
 - Element
 - All the Above



Cascading Style Sheet 3.0

Lesson 2: Working with Text and Fonts

Capgemini Public

Lesson Objectives

- In this lesson, we will learn:
 - Text Formatting
 - Text Effects
 - Fonts



Text Formatting

- Following properties can be specified with the text formatting
 - Text Color
 - Text Alignment
 - Text Decoration
 - Text Transformation
 - Text Indentation
 - Text Shadow
 - Word-wrap

Text Color :The color property is used to set the color of the text.

Text Alignment:The text-align property is used to set the horizontal alignment of a text.

Text can be centered, or aligned to the left or right, or justified.

Text Decoration:The text-decoration property is used to set or remove decorations from text.

The text-decoration property is mostly used to remove underlines from links for design purposes:

Text Transformation:The text-transform property is used to specify uppercase and lowercase letters in a text.

It can be used to turn everything into uppercase or lowercase letters, or capitalize the first letter of each word.

Text Indentation:The text-indentation property is used to specify the indentation of the first line of a text.

Text Shadow: In CSS3, the text-shadow property applies shadow to text

Word Wrapping:In CSS3, the word-wrap property allows you to force the text to wrap - even if it means splitting it in the middle of a word

Text Color

- Color property can be specified as follows:
 - a HEX value - like "#ff0000"
 - an RGB value - like "rgb(255,0,0)"
 - a color name - like "red"
- Example
 - `body {color:blue;}`
 - `h1 {color:#00ff00;}`
 - `h2 {color:rgb(255,0,0);}`

Text Alignment and Text Decoration

- The text-align property is used to set the horizontal alignment of a text.

Example:

- h1 {text-align:center;}
 - p.date {text-align:right;}
 - p.main {text-align:justify;}
- The text-decoration property is used to set or remove decorations from text.

Example:

- h1 {text-decoration:overline;}
- h2 {text-decoration:line-through;}
- h3 {text-decoration:underline;}
- h4 {text-decoration:blink;}

Text Transformation and Text Indentation

- The text-transform property is used to specify uppercase and lowercase letters in a text.
- Example
 - p.uppercase {text-transform:uppercase;}
 - p.lowercase {text-transform:lowercase;}
 - p.capitalize {text-transform:capitalize;}
- The text-indentation property is used to specify the indentation of the first line of a text.
- Example
 - p {text-indent:50px;}

Text Shadow

- In CSS3, the text-shadow property applies shadow to text.
- You specify the horizontal shadow, the vertical shadow, the blur distance, and the color of the shadow:

- Ex: Add a

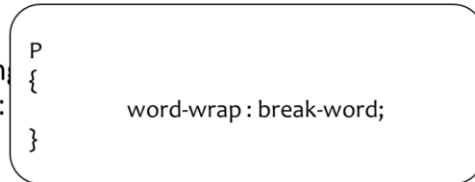
```
h1  
{  
  
}
```

```
text-shadow: 5px 5px 5px #FF0000;
```

Text shadow effect!

Word wrap

- word-wrap property allows you to force the text to wrap - even if it means splitting it in the middle of a word
- Ex:
 - Allow long words to wrap onto the next line:



Capgemini Public

New Text Properties:

[hanging-punctuation](#): Specifies whether a punctuation character may be placed outside the line box

[punctuation-trim](#): Specifies whether a punctuation character should be trimmed

[text-align-last](#): Describes how the last line of a block or a line right before a forced line break is aligned when text-align is "justify"

[text-emphasis](#): Applies emphasis marks, and the foreground color of the emphasis marks, to the element's text

[text-justify](#): Specifies the justification method used when text-align is "justify"

[text-outline](#): Specifies a text outline

[text-overflow](#): Specifies what should happen when text overflows the containing element

[text-shadow](#): Adds shadow to text

[text-wrap](#): Specifies line breaking rules for text

[word-break](#): Specifies line breaking rules for non-CJK scripts

[word-wrap](#): Allows long, unbreakable words to be broken and wrap to the next line

Fonts

- CSS font properties define the font family, boldness, size, and the style of a text.

- **Font-Family : Ex**

```
p{font-family:"Times New Roman", Times, serif;}
```

- **Font**

```
p.normal {font-style:normal;}
p.italic {font-style:italic;}
p.oblique {font-style:oblique;}
```

- **Font**

```
h1 {font-size:40px;}
p {font-size:14px;}
```

Capgemini Public

Font Family:The font family of a text is set with the font-family property.

If the name of a font family is more than one word, it must be in quotation marks, like font-family: "Times New Roman".

More than one font family is specified in a comma-separated list

Font Style:The font-style property is mostly used to specify italic text. This property has three values:

normal - The text is shown normally

italic - The text is shown in italics

oblique - The text is "leaning" (oblique is very similar to italic, but less supported)

Font Size : Setting the text size with pixels gives you full control over the text size:

It can be set either using **px** attribute or **em** attribute as follows:

```
h1 {font-size:40px;}
```

```
h1 {font-size:2.5em;}
```

Note:The em size unit is recommended by the W3C.

1em is equal to the current font size. The default text size in browsers

is 16px. So, the default size of 1em is 16px.

The size can be calculated from pixels to em using this formula: $pixels/16=em$

Demo : Text and Font

- Lesson02
 - demoFontText.html
 - word_wrap.html



Lesson Summary

- In this lesson, you have learnt about
 - Text Formatting
 - Text Effects
 - Fonts



Additional notes for
instructor

Review Questions

Ans-1 : Option 3

Ans-2 : Option 1

- Question 1: Given :

```
h1
{
text-shadow: A ,B ,C,D;
}
```

What property does C represents?

- Option 1: Color
 - Option 2: Vertical Shadow
 - Option 3: Blur
 - Option 4: Horizontal Shadow
- Question 2: Color can be applied to Fonts with CSS 3
 - Option 1: TRUE
 - Option 2: FALSE



Cascading Style Sheet 3.0

Lesson 3: CSS Selectors

Lesson Objectives

- In this lesson, you will be learning about:

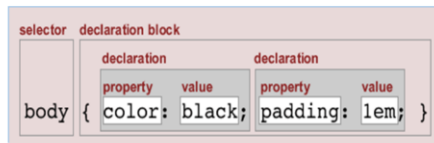
- Introduction to Selectors
- Universal Selector
- Type Selector
- Class Selector
- ID Selector
- Attribute Selector
- Pseudo Classes



Selectors

- Introduction:

- Selectors are one of the most important aspects of CSS as they are used to "select" elements on an HTML page so that they can be styled.
- The selector "selects" the elements on an HTML page that are affected by the rule set.
- A rule or "rule set" is a statement that tells browsers how to render particular elements on an HTML page
- A rule set consists of a selector followed by a declaration block.
- Rule structure



Capgemini Public

Text Color :The color property is used to set the color of the text.

Text Alignment:The text-align property is used to set the horizontal alignment of a text.

Text can be centered, or aligned to the left or right, or justified.

Text Decoration:The text-decoration property is used to set or remove decorations from text.

The text-decoration property is mostly used to remove underlines from links for design purposes:

Text Transformation:The text-transform property is used to specify uppercase and lowercase letters in a text.

It can be used to turn everything into uppercase or lowercase letters, or capitalize the first letter of each word.

Text Indentation:The text-indentation property is used to specify the indentation of the first line of a text.

Text Shadow: In CSS3, the text-shadow property applies shadow to text

Word Wrapping:In CSS3, the word-wrap property allows you to force the text to wrap - even if it means splitting it in the middle of a word

Selectors

- Example

- `h1 { color: blue; margin-top: 1em; }`
- `p { padding: 5px; }`
- `td { background-color: #ddd; }`

Universal Selector

- The universal selector matches any element type.
- Example:

This rule set will be applied to every element in a document

```
* {
  margin : 0;
  padding: 0;
}
```

Capgemini Public

It's important not to confuse the universal selector with a wildcard character—the universal selector doesn't match “zero or more elements.” Consider the following HTML fragment:

```
<body>
<div>
  <h1>The <em>Universal</em> Selector</h1>
  <p>We must <em>emphasize</em> the following:</p>
  <ul>
    <li>It's <em>not</em> a wildcard.</li>
    <li>It matches elements regardless of <em>type</em>.</li>
  </ul>
  This is an <em>immediate</em> child of the division.
</div>
</body>
```

The selector `div * em` will match the following `em` elements:

“Universal” in the `h1` element (* matches the `<h1>`)

“emphasize” in the `p` element (* matches the `<p>`)

“not” in the first `li` element (* matches the `<ul>` or the `<li>`)

“type” in the second `li` element (* matches the `<ul>` or the `<li>`)

However, it won't match the `<em>immediate</em>` element, since that's an immediate child of the `div` element—there's nothing between `<div>` and `<em>` for the `*` to match.

Type selectors

- While the universal selector matches any element, an element type selector matches elements with the corresponding element type name.
- Type selectors are case insensitive in HTML (including XHTML served as text/html), but are case sensitive in XML (including XHTML served as XML).
- Example

```
ul {  
  : declarations  
}
```

- A type selector like the above `ul` matches all the elements within an HTML or XML document that are marked up as follows:
- `<ul> ... </ul>`

The most common and easy to understand selectors are type selectors. Type selectors will select any HTML element on a page that matches the selector, regardless of their position in the document tree. For example:

```
em {color: blue; }
```

This rule will select any `<em>` element on the page and color it blue. As you can see from the document tree diagram below, all `<em>` elements will be colored blue, regardless of their position in the document tree

There are a huge range of elements that you can select using type selectors, which means you can change the appearance of any or every element on your page using only type selectors.

Class Selectors

- Selecting elements on the basis of their class names is a very common technique in CSS
- While type selectors target every instance of an element, class selectors can be used to select any HTML element that has a class attribute, regardless of their position in the document tree.

• Example

```
<body>
<p class="big">This is some <em>text</em></p>
  <p>This is some text</p>
  <ul>
    <li class="big">List item</li>
    <li>List item</li>
    <li>List <em>item</em></li></ul>
</body>
```

```
.big { font-size: 110%; font-weight: bold; }
```

- Above code targets the first paragraph and first list items on a page to make them stand out

Combining class and type selectors:

If you want to be more specific, you can use class and type selectors together. Any type selectors can be used.

```
div.big { color: blue; }
td.big { color: yellow; }
label.big { color: green; }
form.big { color: red; }
```

ID Selector

- An ID selector matches an element that has a specific id attribute value. Since id attributes must have unique values, an ID selector can never match more than one element.
- In its simplest form, an ID selector looks like this:

```
#navigation
{
  : declarations
}
```

- This selector matches any element whose id attribute value is equal to "navigation"

```
#firstname
{
  background-color:yellow;
}
```

Capgemini Public

Code:

```
<!DOCTYPE html>
<html>
<head>
<style>
#firstname
{
    background-color:yellow;
}
</style>
</head>
<body>

<h1>Welcome to My Homepage</h1>

<div class="intro">
<p id="firstname">My name is iGATE.</p>
<p id="hometown">I live in Bangalore.</p>
</div>

<p>My best friend was Patni.</p>

</body>
</html>
```

Attribute Selector

- All HTML elements can have associated properties, called attributes. These attributes generally have values. Any number of attribute/value pairs can be used in an element's tag - as long as they are separated by spaces. They may appear in any order.
- In the example below, the code segments highlighted in blue are attributes and the segments highlighted in red are attribute values

```
<h1 id="section1"/>  
  
<img title="mainimage" alt="main image"/>  
<a href="foo.htm"/>  
<p class="maintext"/>  
<form style="padding: 10px"/>
```

Attribute Selector

- Attribute selectors are used to select elements based on their attributes or attribute value. For example, you may want to select any image on an HTML page that is called "small.gif". This could be done with the rule below, that will only target images with the chosen name:
- There are four types of attribute selectors.

- Example for Select based on attribute

```
img[title] { border: 1px solid #000; }  
img[width] { border: 1px solid #000; }
```

- The example above will select an element (in this case "img") with the relevant attribute

- Example for Select based on value

```
img[src="small.gif"] { border: 1px solid #000; }
```

- The above example selects any image whose attribute (in this case "src") has a value of "small.gif"

Pseudo Classes

- A pseudo-class is similar to a class in HTML, but it's not specified explicitly in the markup. Some pseudo-classes are dynamic—they're applied as a result of user interaction with the document.
- A pseudo-class starts with a colon (:). No whitespace may appear between a type selector or universal selector and the colon, nor can whitespace appear after the colon.

CSS1 introduced the [:link](#), [:visited](#), and [:active](#) pseudo-classes, but only for the HTML a element. These pseudo-classes represented the state of links—unvisited, visited, or currently being selected—in a web page document. In CSS1, all three pseudo-classes were mutually exclusive.

CSS2 expanded the range of pseudo-classes and ensured that they could be applied to any element. [:link](#) and [:visited](#) now apply to any element defined as a link in the document language. While they remain mutually exclusive, the [:active](#) pseudo-class now joins [:hover](#) and [:focus](#) in the group of dynamic pseudo-classes. The [:hover](#) pseudo-class matches elements that are being designated by a pointing device (for example, elements that the user's hovering the cursor over); [:active](#) matches any element that's being activated by the user; and [:focus](#) matches any element that is currently in focus (that is, accepting input).

CSS2 also introduced the [:lang](#) pseudo-class to allow an element to be matched on the basis of its language, and the [:first-child](#) pseudo-class to match an element that's the first child element of its parent.

CSS3 promises an even [greater range of powerful pseudo-classes](#). Remember that pseudo-classes, like [ID selectors](#) and [attribute selectors](#), act like modifiers on [type selectors](#) and the [universal selector](#): they specify additional constraints for the selector pattern, but they don't specify other elements. For instance, the selector `li:first-child` matches a list item that's the first child of its parent; it doesn't match the first child of a list item.

Pseudo Classes

Pseudo class	Description
:link	matches link elements that are unvisited
:visited	matches link elements that have been visited
:active	matches any element that's being activated by the user
:hover	matches elements that are being designated by a pointing device
:focus	matches any element that's currently in focus
:first-child	matches any element that's the first child element of its parent
:lang(C)	allows elements to be matched on the basis of their languages

CSS 3 - Pseudo Classes

Pseudo class	Description
:nth-child(N)	matches elements on the basis of their positions within a parent element's list of child elements
:nth-last-child(N)	matches elements on the basis of their positions within a parent element's list of child elements
:nth-of-type(N)	matches elements on the basis of their positions within a parent element's list of child elements of the same type
:nth-last-of-type(N)	matches elements on the basis of their positions within a parent element's list of child elements of the same type
:last-child	matches an element that's the last child element of its parent element
:first-of-type	matches the first child element of the specified element type
:last-of-type	matches the last child element of the specified element type

CSS 3 - Pseudo Classes

Pseudo class	Description
:only-child	matches an element if it's the only child element of its parent
:only-of-type	matches an element that's the only child element of its type
:root	matches the element that's the root element of the document
:empty	matches elements that have no children
:target	matches an element that's the target of a fragment identifier in the document's URI
:enabled/:disabled	matches user interface elements that are enabled/disabled respectively
:checked Pseudo-class	matches elements like checkboxes or radio buttons that are checked
:not(S)	matches elements that aren't matched by the specified selector

Demo : Selector

- `demoType.html`
- `demoId.html`
- `demoClass.html`
- `demoAttributeSelector.html`
- `demoPseudoClasses.html`



Lesson Summary

- In this lesson, you have learnt about:

- Universal Selector
- Type Selector
- Class Selector
- ID Selector
- Attribute Selector
- PseudoClasses

